

Relatório Técnico: Previsão de Preços de Imóveis com Fusão de Dados e Redes MLP

Felipe Silva Loschi
Engenharia de Software (GES - 601)

21 de maio de 2026

Resumo

Este relatório mostra o desenvolvimento de um modelo de Machine Learning para prever preços de casas usando características físicas do imóvel e dados demográficos da região. Juntando as duas bases de dados através do código postal, criei uma arquitetura de rede neural Multi-Layer Perceptron (MLP) com TensorFlow e Keras. O pipeline foi feito com cuidado para evitar vazamento de dados (*Data Leakage*) e trata problemas não-lineares aplicando transformação logarítmica e uma seleção de variáveis baseada no coeficiente de Spearman e em regras de negócio.

1 Introdução e Fusão de Dados

O objetivo do projeto é criar um modelo capaz de prever o preço de venda de imóveis residenciais. Para deixar o modelo mais inteligente, juntei a base com os dados físicos das casas (`kc_house_data.csv`) com o dataset de informações demográficas da região (`zipcode_demographics.csv`), usando a coluna `zipcode` como chave para o *merge*.

Colunas que serviam apenas como identificadores ou dados temporais que não ajudariam na predição (`id` e `date`) foram retiradas do dataset. Além disso, criei uma nova variável binária chamada `was_renovated` a partir do ano de reforma (`yr_renovated`) para indicar simplesmente se a casa foi reformada ou não, simplificando essa informação para a rede neural.

2 Análise Exploratória de Dados (EDA) e Seleção de Atributos

Na análise exploratória, estudei como as variáveis preditoras se distribuem e como elas se relacionam com o preço (`price`). Como muitos dados não seguem uma distribuição linear perfeita e possuem assimetria, usei o coeficiente de correlação de **Spearman** para guiar a escolha das melhores variáveis.

2.1 Identificação de Outliers e Filtragem

Durante a EDA, encontrei um erro claro na coluna de quantidade de quartos (`bedrooms`): um dos imóveis estava registrado com mais de 30 quartos, o que não fazia sentido para o tamanho da casa. Fiz um filtro para remover esse outlier e manter apenas os registros reais (< 30 quartos), evitando que esse dado inconsistente atrapalhasse o treino do modelo.

2.2 Critério de Seleção de Atributos

Para não sobrecarregar a rede MLP com variáveis irrelevantes ou redundantes, defini um critério de corte: selecionei apenas as variáveis que tinham uma correlação em módulo igual ou maior que

0,3 ($|\rho| \geq 0.3$) com a variável alvo **price**. Esse valor é um bom limite prático para garantir que estamos usando apenas variáveis que realmente trazem alguma informação útil para o modelo.

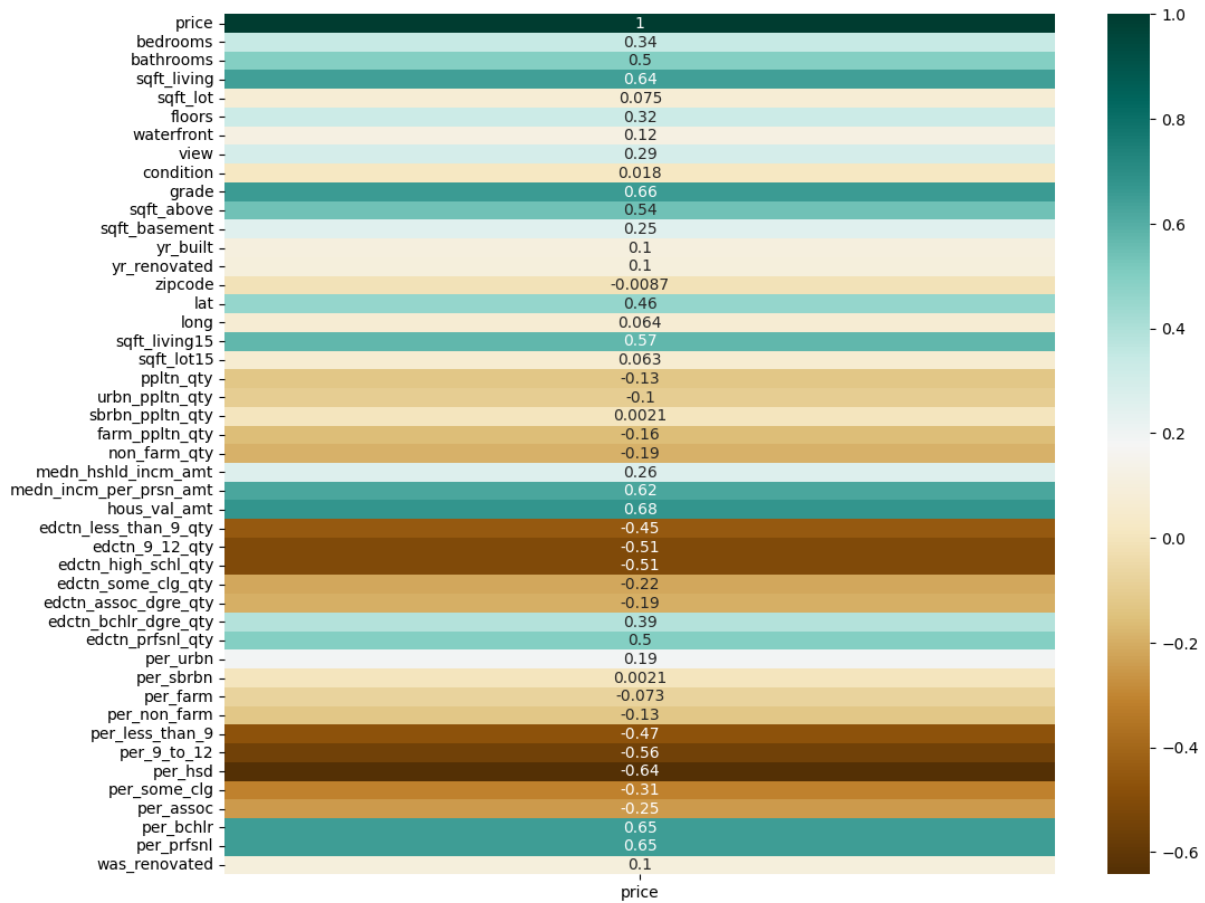


Figura 1: Matriz de Correlação de Spearman filtrada para a variável dependente Price.

Apesar de seguir esse corte matemático, abri duas exceções importantes pensando na lógica do negócio:

- **view:** A variável que mede a qualidade da vista da casa teve correlação de 0,29. Como ficou muito perto do corte de 0,3 e a vista é um fator que pesa muito na decisão de quem vai comprar um imóvel, decidi manter essa coluna no modelo.
- **long:** Mantive a longitude para fazer par com a latitude (**lat**). Se eu tirasse a longitude, o modelo perderia a noção de geolocalização exata, ficando impossibilitado de entender quais bairros ou regiões específicas são mais valorizados.

Também retirei as colunas com a contagem absoluta de escolaridade (**edctn_*_qty**) deixando apenas as respectivas variáveis percentuais, que estão em uma escala comum a todos. No final, o modelo ficou com 17 variáveis preditoras.

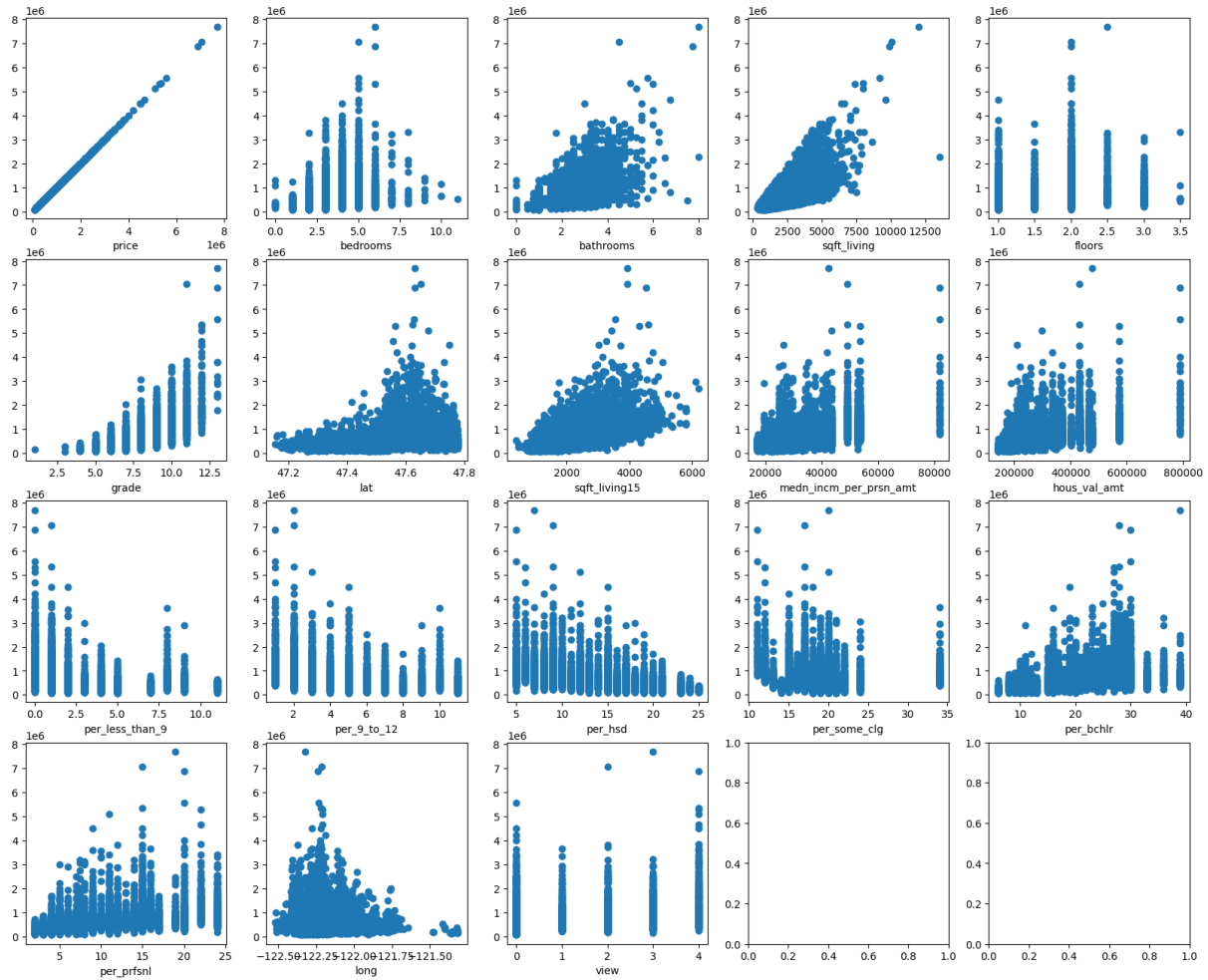


Figura 2: Grid de dispersão das 17 features selecionadas após o corte de correlação e ajustes de domínio.

2.3 Transformação Logarítmica

Notei que o preço (`price`) e outras variáveis financeiras/demográficas, como renda (`medn_incm_per_prsn_amt`) e valor médio de moradia regional (`hous_val_amt`), tinham distribuições muito esticadas para a direita (presença de valores muito altos). Para resolver isso e ajudar o algoritmo a convergir melhor, apliquei a transformação logarítmica natural:

$$\tilde{x} = \ln(x) \quad (1)$$

Essa transformação ajuda a deixar os dados em uma escala mais equilibrada, facilitando o ajuste dos pesos da rede neural durante o treino.

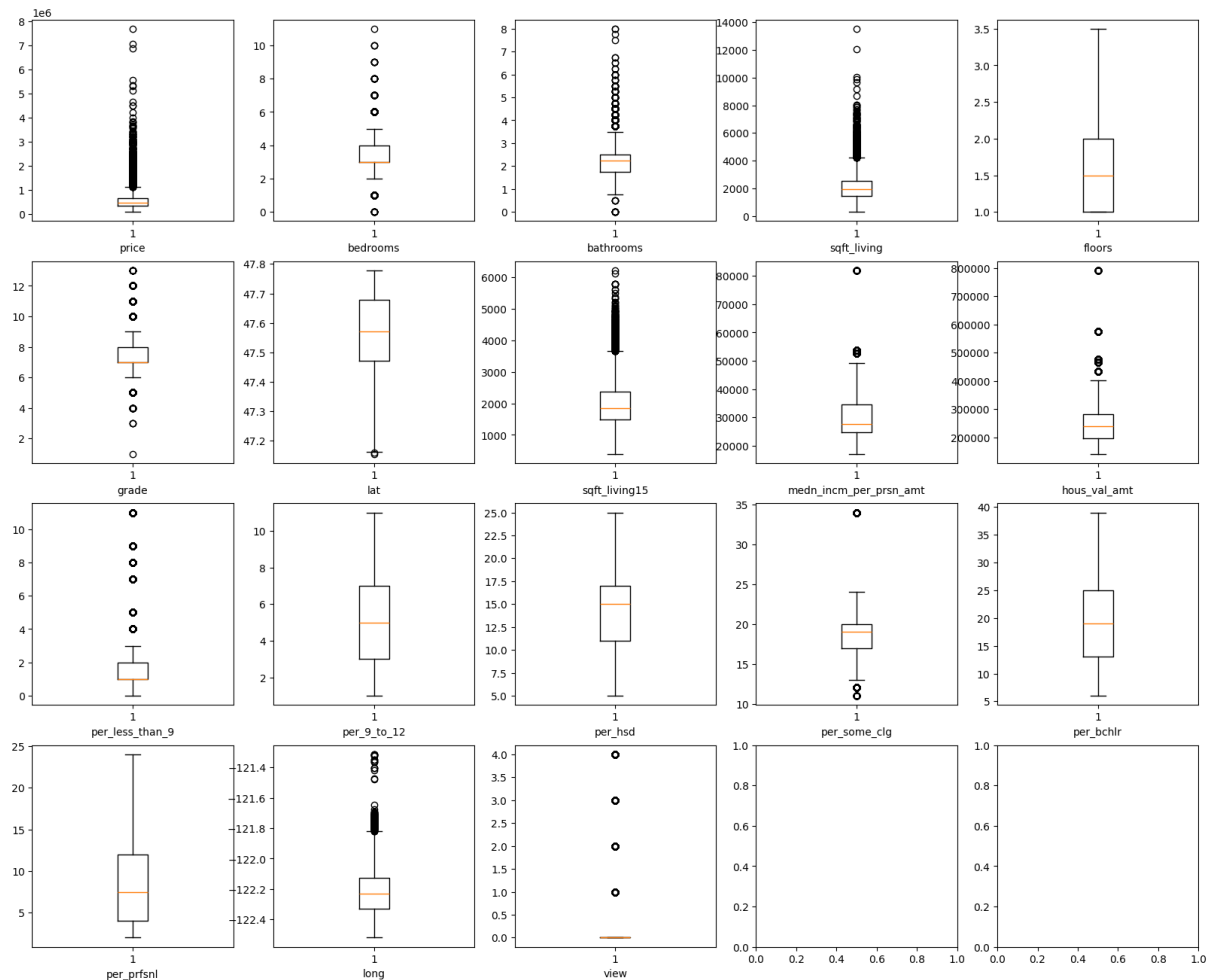


Figura 3: Análise de Dispersão e Limites de Outliers via Boxplots das Features Seleccionadas.

3 Engenharia de Recursos e Pipeline de Preparação

3.1 Prevenção de Data Leakage

Para garantir que o modelo funcione bem com dados novos e do mundo real (`future_unseen_examples.csv`), dividi a base em 80% para treino e 20% para teste. Na hora de padronizar os dados com o `StandardScaler`, fiz o processo do jeito correto para evitar o vazamento de dados (*Data Leakage*): o ajuste (*fit*) foi feito apenas no conjunto de treino, usando apenas o *transform* nos dados de teste e validação:

```
1 # Isolamento estrito do Scaler para evitar Data Leakage
2 standard = StandardScaler()
3 X_train = standard.fit_transform(X_train)
4 X_test = standard.transform(X_test)
```

4 Arquitetura da Rede Neural MLP

O modelo escolhido foi uma rede neural Multi-Layer Perceptron (MLP) criada com Keras e TensorFlow. Montei uma estrutura com camadas densas que vão diminuindo de tamanho para extrair os padrões dos dados de forma eficiente.

Tabela 1: Especificação Arquitetural da Rede MLP Dedicada

Camada	Neurônios / Unidades	Função de Ativação
Input (Entrada)	17	—
Dense 1	256	ReLU
Dense 2	128	ReLU
Dense 3	64	ReLU
Dense 4	32	ReLU
Saída (Output)	1	Linear (Regressão)

Usei a função de perda de Erro Quadrático Médio (MSE) e o otimizador **Adam** com a taxa de aprendizado padrão ($\alpha = 10^{-3}$). Nas camadas escondidas usei a função de ativação ReLU para evitar problemas com o sumiço do gradiente, e na última camada usei ativação linear, que é o padrão para problemas de regressão.

5 Análise de Resultados e Generalização

Para evitar o problema de *overfitting* e garantir a generalização do modelo, usei o *train_test_split*, dividindo o conjunto em 80% para treino e 20% para teste. Além disso, usei o *Early Stopping* para monitorar o erro de um conjunto de validação (20% separado do treino), configurado com uma paciência de 15 épocas.

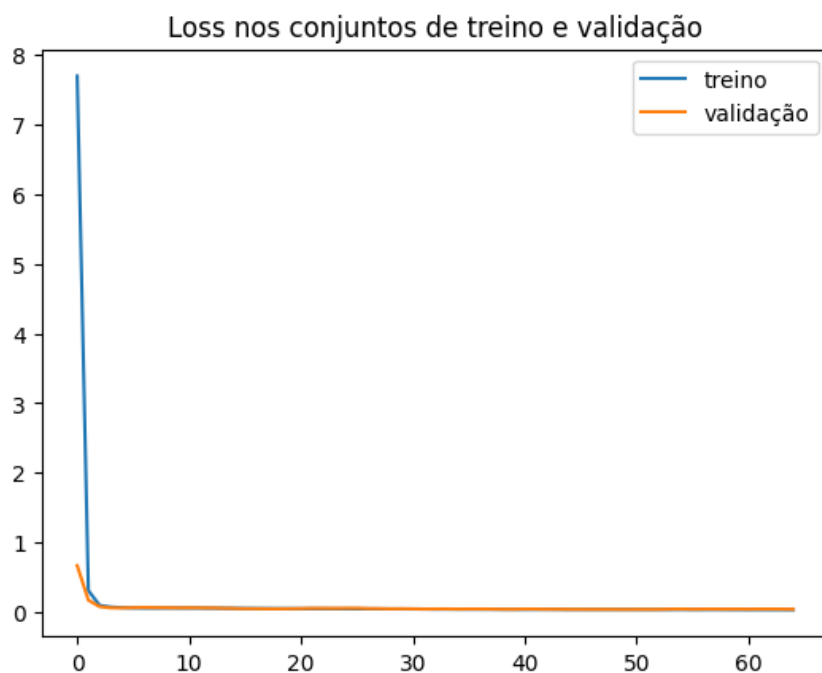


Figura 4: Histórico de Treinamento: Convergência das funções de perda no Treino e Validação.

O comportamento das curvas de erro de treino e validação caindo juntas mostra que o modelo aprendeu os padrões de forma estável. No fim das contas, isso significa que o modelo está pronto para generalizar bem e pronto para ser testado na prática para estimar preços de imóveis de forma confiável.